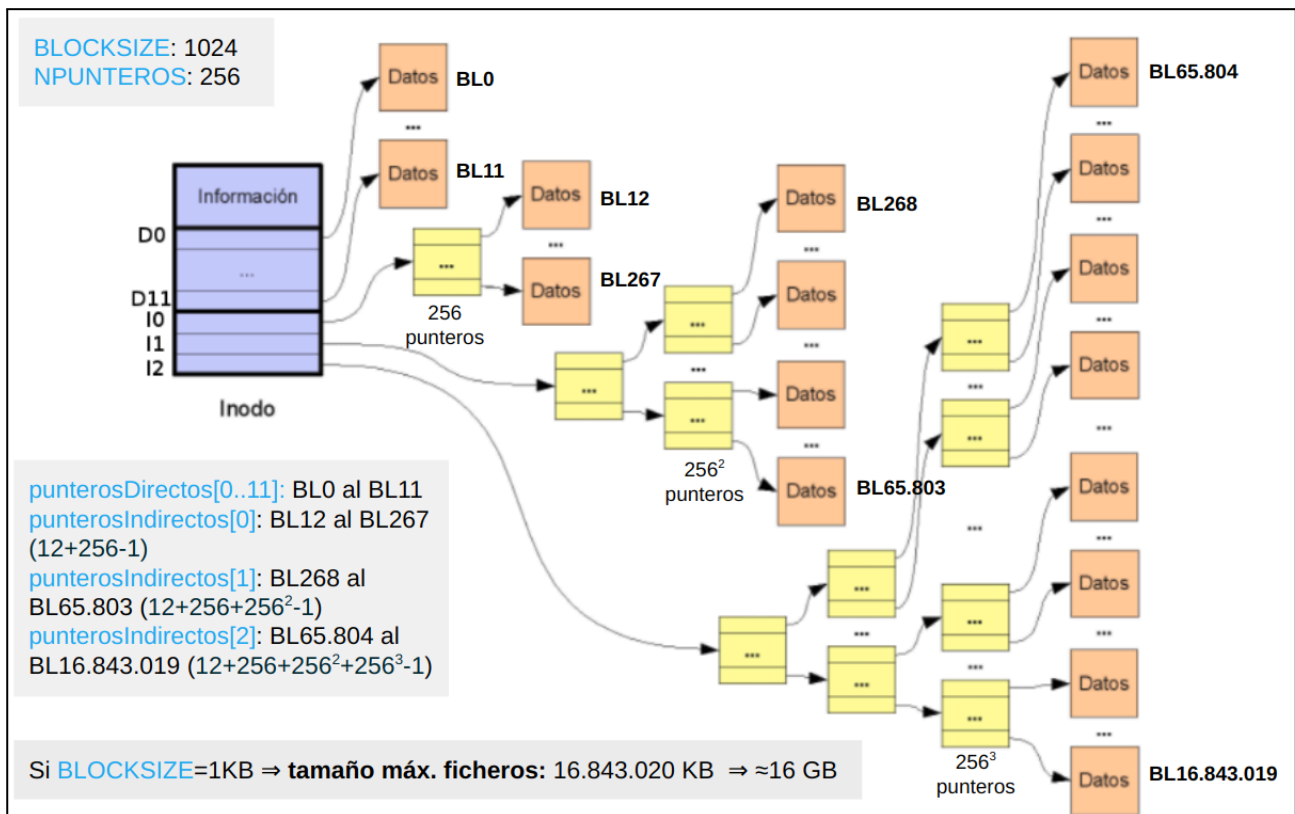


Nivel 4: `ficheros_basico.c` {`traducir_bloque_inodo()` y auxiliares: `obtener_indice()` y `obtener_nRangoBL()`}

Antes de nada, repasemos cómo los inodos apuntan a los bloques de datos:



En concreto, tenemos inodos con 12 **punteros directos** a bloques físicos de datos y 3 **punteros indirectos**, de diferentes niveles, que apuntan a bloques de punteros, y suponemos que el tamaño de bloque es de 1.024 bytes (por lo que en un bloque caben 256 punteros ya que cada puntero se representa con 4 bytes correspondientes al `sizeof(unsigned int)`).

Entonces:

- Los **punteros directos** a bloques físicos de datos, `punterosDirectos[0]` .. `punterosDirectos[11]`¹, permiten encontrar donde se hallan físicamente los primeros 12 bloques lógicos del inodo, es decir, los comprendidos entre el 0 y el 0+12-1: o sea del 0 al 11.

¹ D0..D11 para abreviar

- El **puntero de bloques indirectos 0**, `punterosIndirectos[0]`², permite encontrar donde se hallan físicamente los siguientes 256 bloques lógicos del inodo, es decir, los comprendidos entre el 0+12 y el 0+12+256-1: o sea del 12 al 267.
- El **puntero de bloques indirectos 1**, `punterosIndirectos[1]`³, permite encontrar donde se hallan físicamente los siguientes 256² (65.536) bloques lógicos del inodo, es decir, los comprendidos entre el 0+12+256 y el 0+12+256+256²-1: o sea del 268 al 65.803.
- El **puntero de bloques indirectos 2**, `punterosIndirectos[2]`⁴, permite encontrar donde se hallan físicamente los siguientes 256³ (16.777.216) bloques lógicos del inodo, es decir, los comprendidos entre el 0+12+256+256² y el 0+12+256+256²+256³-1: o sea del 65.804 al 16.843.019.

A partir de los datos anteriores podemos crear una función auxiliar `obtener_nRangoBL()` que asocie un **rango** para cada conjunto de bloques lógicos:

- `nRangoBL=0` para bloques lógicos [0 , 11],
- `nRangoBL=1` para bloques lógicos [12 , 267],
- `nRangoBL=2` para bloques lógicos [268 , 65.803] y
- `nRangoBL=3` para bloques lógicos [65.804 , 16.843.019].

También ha de actualizar una variable puntero, `ptr`, pasada por referencia, para que apunte donde lo hace el puntero correspondiente del inodo.

Primeramente podemos definir constantes simbólicas (en `ficheros_basico.h`) que nos ayuden a determinar los rangos de punteros⁵:

```
#define NPUNTEROS (BLOCKSIZE / sizeof(unsigned int)) // 256 punteros por bloque
#define DIRECTOS 12
#define INDIRECTOS0 (NPUNTEROS + DIRECTOS) // 268
#define INDIRECTOS1 (NPUNTEROS * NPUNTEROS + INDIRECTOS0) // 65.804
#define INDIRECTOS2 (NPUNTEROS * NPUNTEROS * NPUNTEROS + INDIRECTOS1) // 16.843.020
```

Y luego (en `ficheros_basico.c`) definimos la función `obtener_nRangoBL()` para obtener el **rango de punteros** en el que se sitúa el bloque lógico que buscamos (0:D, 1:I0, 2:I1, 3:I2), y **obtenemos además la dirección almacenada en el puntero correspondiente del inodo**, `ptr`:

```
funcion obtener_nRangoBL (*inodo: struct inodo, nblogico: unsigned ent, *ptr: unsigned ent)
devolver nrangoBL: ent

    si nblogico < DIRECTOS entonces // < 12
        *ptr = inodo->punterosDirectos[nblogico]
```

² I0 para abreviar

³ I1 para abreviar

⁴ I2 para abreviar

⁵ Muy importante el uso de los paréntesis en estas definiciones!!!

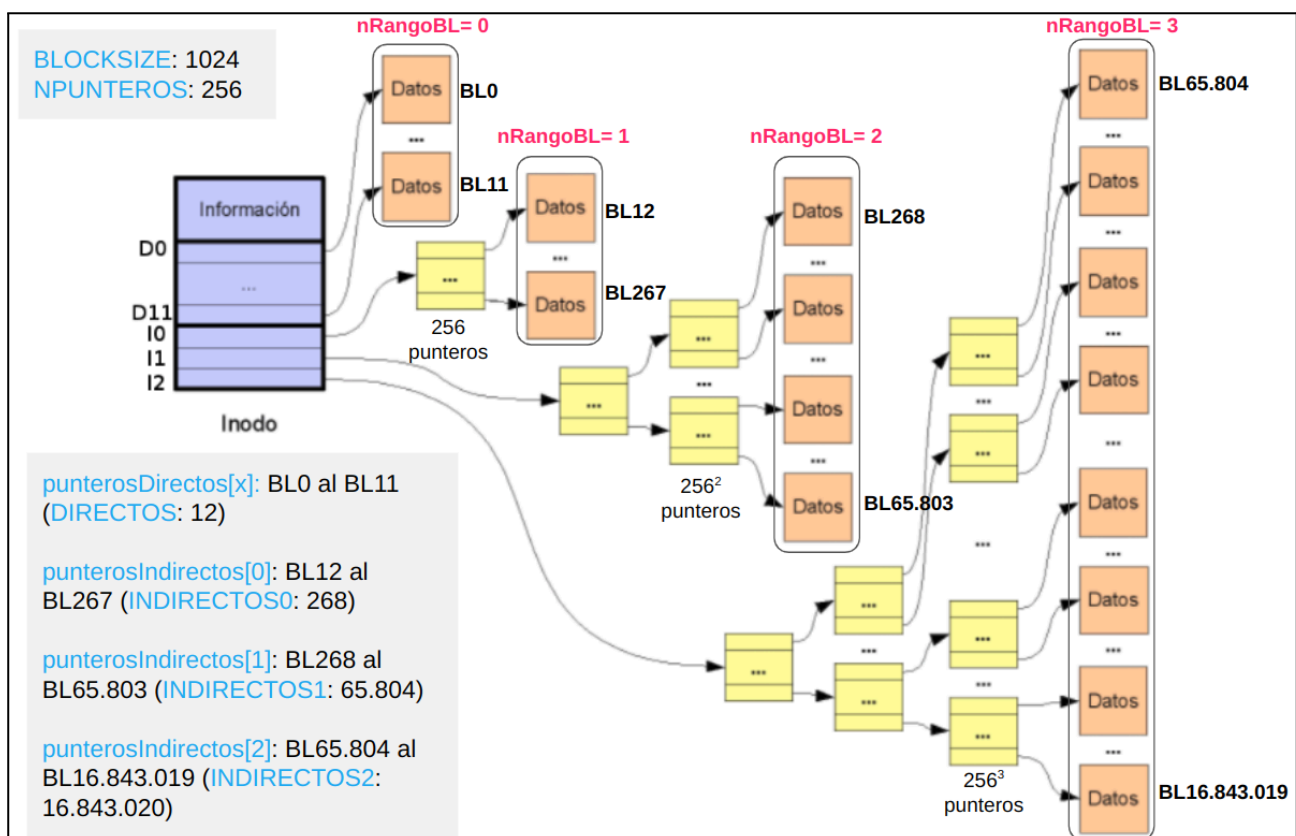
Sistema de ficheros

Nivel 4

`ficheros_basico.c` {traducir_bloque_inodo(),
obtener_indice(), obtener_nRangoBL()}

```

devolver 0
si_no si nblogico<INDIRECTOS0 entonces // <268
    *ptr:=inodo->punterosIndirectos[0]
    devolver 1
si_no si nblogico<INDIRECTOS1 entonces // <65.804
    *ptr:=inodo->punterosIndirectos[1]
    devolver 2
si_no si nblogico<INDIRECTOS2 entonces // <16.843.020
    *ptr:=inodo->punterosIndirectos[2]
    devolver 3
si_no
    *ptr:=0
    error("Bloque lógico fuera de rango")
    devolver -1
fsi
ffuncion
    
```



Veamos ahora ejemplos de cómo se obtienen los índices de los bloques de punteros:

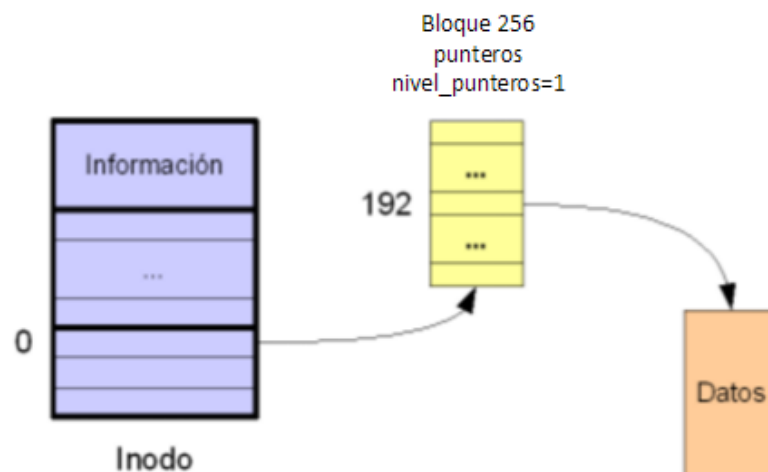
Si, por ejemplo, queremos traducir el **bloque lógico número 204**:

Como $12 \leq 204 < 268$ (**nRangoBL=1**), hemos de recurrir al **punterosIndirectos[0]**.

Restamos los 12 punteros anteriores para obtener un valor índice dentro del rango de los 256 punteros, [0,255], del bloque de índices de nivel 1 que cuelga de `punterosIndirectos[0]`:

$$204 - 12 = 192 \ (\subset [0,255])$$

Por tanto el número de nuestro bloque físico se encuentra en el puntero número **192** del bloque de índices de nivel 1 apuntado por `punterosIndirectos[0]`.



Si, por ejemplo, queremos traducir el **bloque lógico número 30.004**:

Como $268 \leq 30.004 < 65.804$ ($nRangoBL=2$), hemos de recurrir a `punterosIndirectos[1]`.

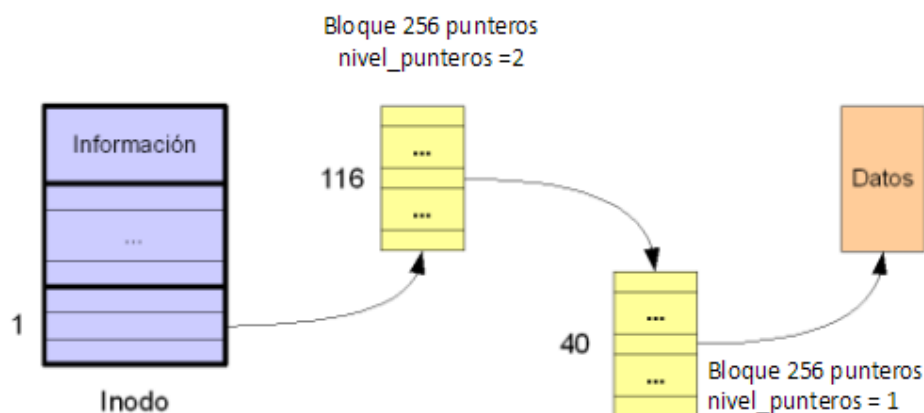
Restamos los 268 punteros anteriores ($12+256$) para obtener un valor dentro del rango de los 256^2 (65.536) bloques que puedo direccionar con la estructura global de punteros que cuelga de `punterosIndirectos[1]`:

$$30.004 - 268 = 29.736 \ (\subset [0, 65.535])$$

Dado que ese valor puede estar en uno de los 256 bloques de punteros de nivel 1, que cuelgan del bloque de punteros de nivel 2, tendremos que dividir ese valor entre 256 para obtener de cuál se trata (índice en el bloque de **punteros de nivel 2**). El resto de la división (módulo) nos dará el índice correspondiente en ese bloque de **punteros de nivel 1**, apuntado por el que hemos obtenido en el nivel 2:

$$\begin{aligned} 29.736 / 256 &= 116 \\ 29.736 \% 256 &= 40 \end{aligned}$$

Por tanto, el número de nuestro bloque físico se encuentra en el puntero número **40** del bloque apuntado por el puntero número **116** del bloque apuntado por `punterosIndirectos[1]`.



Si, por ejemplo, queremos traducir el **bloque lógico número 400.004**:

Como $65.804 \leq 400.004 < 16.843.020$ ($nRangoBL=3$), hemos de recurrir a [punterosIndirectos\[2\]](#).

Restamos los 65.804 punteros anteriores ($12+256+65.536$) para obtener un valor dentro del rango de los 256^3 ($16.777.216$) bloques que puedo direccionar con la estructura global de punteros que cuelga de [punterosIndirectos\[2\]](#):

$$400.004 - 65.804 = 334.200 (\subset [0, 16.777.215])$$

Para saber ese bloque a qué índice de nivel 3 corresponde, tendremos que dividirlo entre los 256^2 que salen de su estructura:

$$334.200 / 65.536 = 5$$

El resto de esa división habrá que dividirlo entre los 256 bloques de punteros que cuelgan para determinar el índice del bloque de punteros de nivel 2, y el módulo nos dará el índice del nivel 1:

$$334.200 \% 65.536 = 6.520$$

$$6.520 / 256 = 25$$

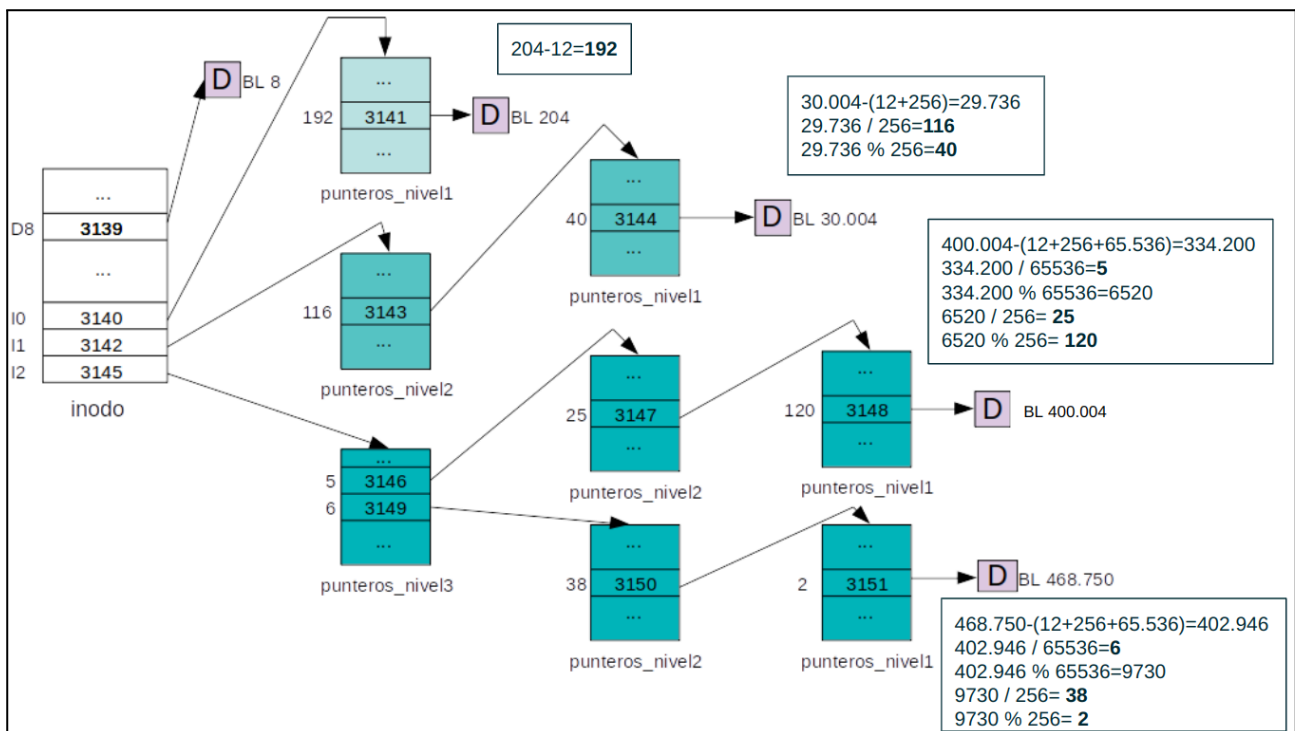
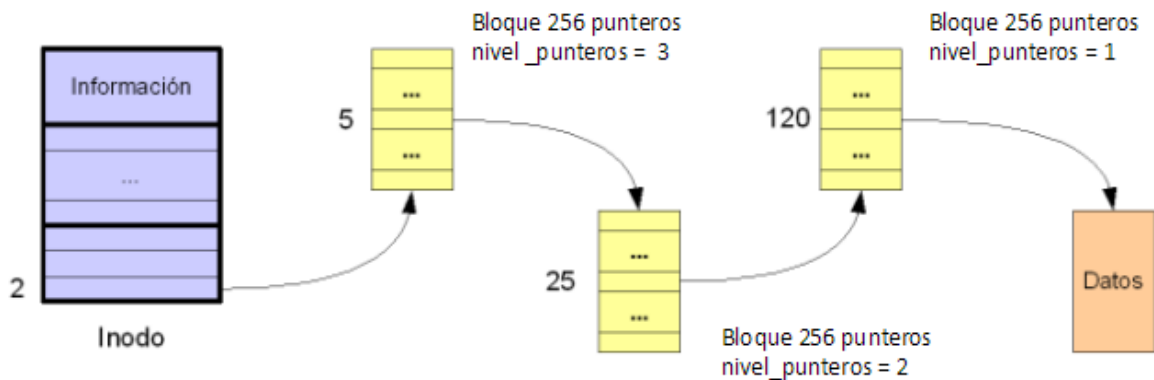
$$6.520 \% 256 = 120$$

Por tanto, el número de nuestro bloque físico se encuentra en el puntero número **120** del bloque apuntado por el puntero número **25** del bloque apuntado por el puntero número **5** del bloque apuntado por indirectos 2.

Sistema de ficheros

Nivel 4

`ficheros_basico.c` {traducir_bloque_inodo(),
obtener_indice(), obtener_nRangoBL()}



De los ejemplos anteriores podemos deducir una función, `obtener_indice()` para generalizar la obtención de los índices de los bloques de punteros⁶:

```
funcion obtener_indice (nblogico: unsigned ent, nivel_punteros: ent) devolver ind: ent
si nblogico < DIRECTOS entonces devolver nblogico //ej. nblogico=8
si_no si nblogico < INDIRECTOS0 entonces devolver nblogico - DIRECTOS //ej. nblogico=204
si_no si nblogico < INDIRECTOS1 entonces //ej. nblogico=30.004
    si nivel_punteros = 2 entonces
        devolver (nblogico - INDIRECTOS0) / NPUNTEROS
    si_no si nivel_punteros=1 entonces
```

⁶ Ver Anexo para un enfoque de descomposición polinomial

```
    devolver (nblogico - INDIRECTOS0) % NPUNTEROS
fsi
si_no si nblogico < INDIRECTOS2 entonces //ej. nblogico=400.004
    si nivel_punteros = 3 entonces
        devolver (nblogico - INDIRECTOS1) / (NPUNTEROS * NPUNTEROS)
    si_no si nivel_punteros = 2 entonces
        devolver ((nblogico - INDIRECTOS1) % (NPUNTEROS * NPUNTEROS)) / NPUNTEROS
    si_no si nivel_punteros = 1 entonces
        devolver ((nblogico - INDIRECTOS1) % (NPUNTEROS * NPUNTEROS)) % NPUNTEROS
fsi
fsi
ffuncion
```

Ahora ya podemos desarrollar la función de `traducir_bloque_inodo()`:

1) `int traducir_bloque_inodo(unsigned int inodo, unsigned int nblogico, unsigned char reservar);`

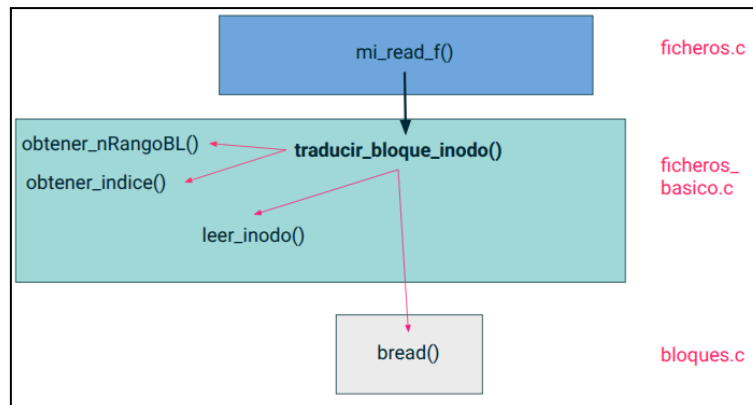
Esta función se encarga de obtener el **nº de bloque físico** correspondiente a un **bloque lógico** determinado del inodo indicado. Enmascara la gestión de los diferentes rangos de punteros directos e indirectos del inodo, de manera que funciones externas no tienen que preocuparse de cómo acceder a los bloques físicos apuntados desde el inodo.

Para desarrollarla, se puede optar tanto por una versión recursiva como por una versión iterativa y utilizar las funciones auxiliares para obtener el **nº de rango de punteros**, el **puntero correspondiente del inodo** y los **índices de los bloques de punteros**.

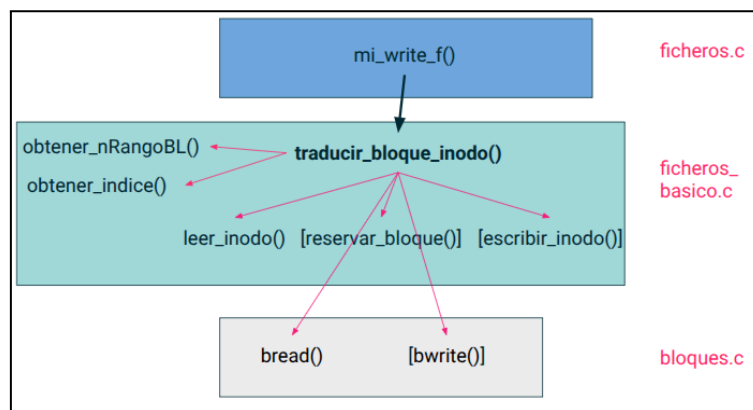
La misma función nos puede servir tanto para sólo **consultar** (al ser llamada por `mi_read_f()` de la capa de ficheros) como para consultar y reservar un bloque libre si ningún bloque físico es apuntado por el número de bloque lógico (al ser llamada por `mi_write_f()` de la capa de ficheros).

Podemos indicar este comportamiento con el argumento `reservar`. De manera genérica:

- Si `reservar` vale **0** utilizaremos `traducir_bloque_inodo()` únicamente para **consultar**:
 - Si **existe** bloque físico de datos, la función devolverá su **posición**.
 - Si **no existe** bloque físico de datos, dará **error** (*¡pero no lo notificamos por pantalla!*).
- Si `reservar` vale **1** utilizaremos `traducir_bloque_inodo()` para consultar y, si no existe bloque físico, también para **reservar**:
 - Si **existe** bloque físico de datos, la función devolverá su **posición**.
 - Si **no existe** bloque físico de datos, lo **reservará** y se devolverá su **posición**.



Esquema de llamadas hacia/desde `traducir_bloque_inodo()` con `reservar=0`.



Esquema de llamadas hacia/desde `traducir_bloque_inodo()` con `reservar=1`.

Hay que tener en cuenta si existen o no los **bloques de punteros intermedios** que precisemos atravesar hasta llegar al **bloque de datos**. En caso de que alguno/s no exista/n:

- si estamos en modo de **lectura**, remitiremos el “**error**” (**sin notificarlo por pantalla**),
- y si estamos en modo **escritura**, habrá que **reservarlos** (inicializando previamente un buffer con 0s para indicar que no apuntan a nada) y **enlazarlos** adecuadamente.

Cada vez que reservemos un bloque (sea para datos o para punteros) habrá que incrementar el campo de **número de bloques ocupados** del inodo.

Si se alteran los punteros del inodo y/o la cantidad de bloques ocupados hay que actualizar el **ctime**!

Un **ejemplo** de [pseudocódigo](#) para la **función optimizada** sería:


```
var
  ptr, ptr_ant, salvar_inodo: unsigned ent
  nRangoBL, nivel_punteros, indice: ent
  buffer[NPUNTEROS]: unsigned ent
  inodo: struct inodo
fvar
  ptr := 0, ptr_ant := 0, salvar_inodo := 0, indice := 0
  leer_inodo(ninodo, &inodo)
  nRangoBL := obtener_nRangoBL(&inodo, nblogico, &ptr); //0:D, 1:I0, 2:I1, 3:I2
  nivel_punteros := nRangoBL //el nivel_punteros +alto es el que cuelga directamente del inodo

si nRangoBL=0 entonces //Caso punteros Directos

  si ptr=0 //no existe bloque de datos
  si reservar=0 entonces devolver -1 fsi //error ÷ bloque -> no imprimir error por pantalla!!!
  ptr = reservar_bloque() //de datos
  inodo.numBloquesOcupados++
  inodo.ctime = time(NULL)
  inodo.punterosDirectos[nblogico] := ptr //asignamos la dirección del bl. de datos en el inodo
  salvar_inodo := 1
  fsi

si_no //Caso de punteros Indirectos
  mientras nivel_punteros>0 hacer //iterar para cada nivel de punteros indirectos
  si ptr=0 entonces //no cuelgan bloques de punteros
  si reservar=0 entonces devolver -1 fsi// error ÷ bloque -> no imprimir error por pantalla!!!

  //reservar bloques de punteros y crear enlaces desde el inodo hasta el bloque de datos
  ptr := reservar_bloque() //de punteros
  inodo.numBloquesOcupados++
  inodo.ctime = time(NULL) //fecha actual
  salvar_inodo := 1

  si nivel_punteros = nRangoBL entonces //el bloque cuelga directamente del inodo
    inodo.punterosIndirectos[nRangoBL-1] := ptr
  si_no //el bloque cuelga de otro bloque de punteros
    buffer[indice] := ptr
    bwrite(ptr_ant, buffer) //salvamos en el dispositivo el buffer de punteros modificado
  fsi
  memset(buffer, 0, BLOCKSIZE) //ponemos a 0 todos los punteros del buffer
  si_no //ptr!=0
    bread(ptr, buffer) //leemos del dispositivo el bloque de punteros ya existente
  fsi
  indice := obtener_indice(nblogico, nivel_punteros)
  ptr_ant := ptr //guardamos el puntero actual
  ptr := buffer[indice] // y lo desplazamos al siguiente nivel
  nivel_punteros--
  fmientras //al salir de este bucle ya estamos al nivel de datos
```

```
si ptr=0 entonces
  si reservar=0 entonces devolver -1 fsi //bloque inexistente -> no imprimir error por pantalla!!!
  ptr := reservar_bloque() //de datos
  inodo.numBloquesOcupados++
  inodo.ctime = time(NULL) //fecha actual
  salvar_inodo := 1

  buffer[indice] := ptr //asignamos la dirección del bloque de datos en el buffer
  bwrite(ptr_ant, buffer) //salvamos en el dispositivo el buffer de punteros modificado
fsi
fsi //fin caso Indirectos

//salvar el inodo si se han hecho cambios y se desea no tener un big lock al usar semáforos
si salvar_inodo entonces escribir_inodo(ninodo, &inodo) fsi

devolver ptr //nº de bloque físico correspondiente al bloque de datos lógico, nblogico
ffuncion
```

Recomendación:

- Haced el seguimiento a mano del algoritmo (con `reservar=1`) para los siguientes bloques lógicos de un mismo inodo: 8, 204, 30.004, 400.004, y 468.750, suponiendo que estamos en la situación inicial de que todos los punteros del inodo apuntan a 0 y no hay nada escrito en la zona de datos del dispositivo virtual. Comprobad los índices obtenidos para cada nivel con los ejemplos proporcionados, y observad cómo se va creando la estructura multinivel de punteros.

Tests de prueba

En este nivel `leer_sf.c` ya **no hay que reservar y liberar un bloque, ni mostrar el mapa de bits, ni el directorio raíz** (no borréis las sentencias, tan sólo ponedlas como comentarios o desactivad su impresión usando `#if DEBUGNx`).

Ahora habrá que añadir a `leer_sf.c` las instrucciones necesarias para **comprobar la traducción de bloques de diferentes rangos de punteros**.

Para ello necesitaremos previamente **reservar artificialmente un inodo** con la función `reservar_inodo()`, y haremos llamadas a la función `traducir_bloque_inodo()` para traducir los bloques lógicos siguientes: 8, 204, 30.004, 400.004 y 468.750.

Mensajes de debugging: En la función `traducir_bloque_inodo()` de `ficheros_basico.c` hay que poner los `fprintf()` necesarios, justo después de la asignación de un valor a `ptr`, en las 4 situaciones posibles:

- Reservar un **bloque de datos** que cuelgue de `punterosDirectos[x]`.
- Reservar un **bloque de datos** que cuelgue de un **bloque de punteros**.
- Reservar un **bloque de punteros** que cuelgue de `punterosIndirectos[x]`.
- Reservar un **bloque de punteros** que cuelgue de otro **bloque de punteros**.

```
$ ./mi_mkfs disco 100000
$ ./leer_sf disco
```

DATOS DEL SUPERBLOQUE

```
posPrimerBloqueMB = 1
posUltimoBloqueMB = 13
posPrimerBloqueAI = 14
posUltimoBloqueAI = 3138
posPrimerBloqueDatos = 3139
posUltimoBloqueDatos = 99999
posInodoRaiz = 0
posPrimerInodoLibre = 1
cantBloquesLibres = 96861
cantInodosLibres = 24999
totBloques = 100000
totInodos = 25000
```

INODO 1. TRADUCCION DE LOS BLOQUES LOGICOS 8, 204, 30.004, 400.004 y 468.750

```
[traducir_bloque_inodo()→ inodo.punterosDirectos[8] = 3139 (reservado BF 3139 para BL 8)]

[traducir_bloque_inodo()→ inodo.punterosIndirectos[0] = 3140 (reservado BF 3140 para punteros_nivel1)]
[traducir_bloque_inodo()→ punteros_nivel1 [192] = 3141 (reservado BF 3141 para BL 204)]

[traducir_bloque_inodo()→ inodo.punterosIndirectos[1] = 3142 (reservado BF 3142 para punteros_nivel2)]
[traducir_bloque_inodo()→ punteros_nivel2 [116] = 3143 (reservado BF 3143 para punteros_nivel1)]
[traducir_bloque_inodo()→ punteros_nivel1 [40] = 3144 (reservado BF 3144 para BL 30004)]

[traducir_bloque_inodo()→ inodo.punterosIndirectos[2] = 3145 (reservado BF 3145 para punteros_nivel3)]
[traducir_bloque_inodo()→ punteros_nivel3 [5] = 3146 (reservado BF 3146 para punteros_nivel2)]
[traducir_bloque_inodo()→ punteros_nivel2 [25] = 3147 (reservado BF 3147 para punteros_nivel1)]
[traducir_bloque_inodo()→ punteros_nivel1 [120] = 3148 (reservado BF 3148 para BL 400004)]

[traducir_bloque_inodo()→ punteros_nivel3 [6] = 3149 (reservado BF 3149 para punteros_nivel2)]
[traducir_bloque_inodo()→ punteros_nivel2 [38] = 3150 (reservado BF 3150 para punteros_nivel1)]
[traducir_bloque_inodo()→ punteros_nivel1 [2] = 3151 (reservado BF 3151 para BL 468750)]
```

DATOS DEL INODO RESERVADO 1

tipo: f

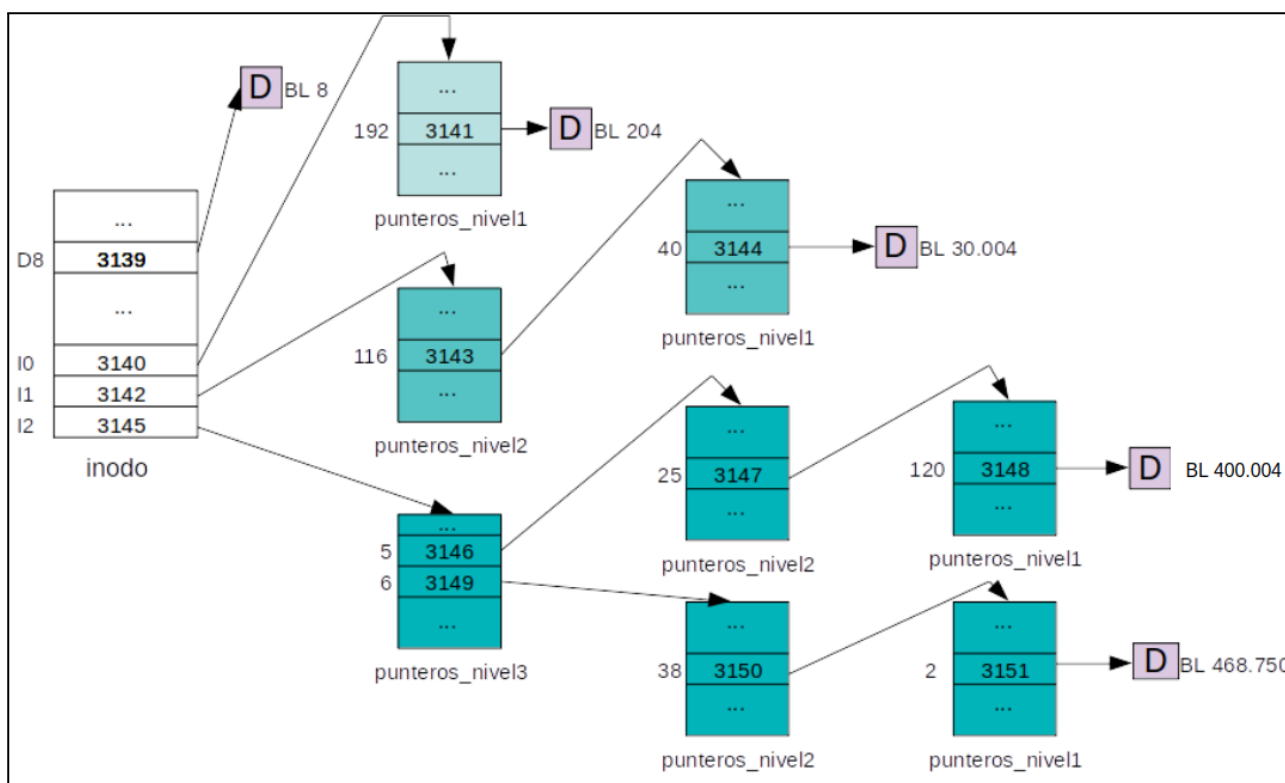
Sistema de ficheros

Nivel 4

`ficheros_basico.c` {traducir_bloque_inodo(),
obtener_indice(), obtener_nRangoBL()}

permisos: 6
atime: Wed 2021-03-10 17:09:42
mtime: Wed 2021-03-10 17:09:42
ctime: Wed 2021-03-10 17:09:42
btime: Wed 2021-03-10 17:09:42
nlinks: 1
tamEnBytesLog: 0
numBloquesOcupados: 13⁷
SB.posPrimerInodoLibre = 2

Los siguientes gráficos ilustran esta ejecución:



⁷ Los ha tenido que reservar porque no existían para el inodo, pero no contienen nada. Para hacer esta prueba estamos llamando a `traducir_bloque_inodo()` con `reservar=1` para ver que cree bien los punteros. Después **desactivaremos** en `leer_sf.c` esas llamadas ya que en la práctica sólo tiene sentido llamar a `traducir_bloque_inodo()` con `reservar=1` cuando vayamos a escribir contenido en los ficheros o a crear entradas en los directorios, ambas acciones desde una capa superior de nuestra biblioteca mediante la función `mi_write_f()`.

Sistema de ficheros

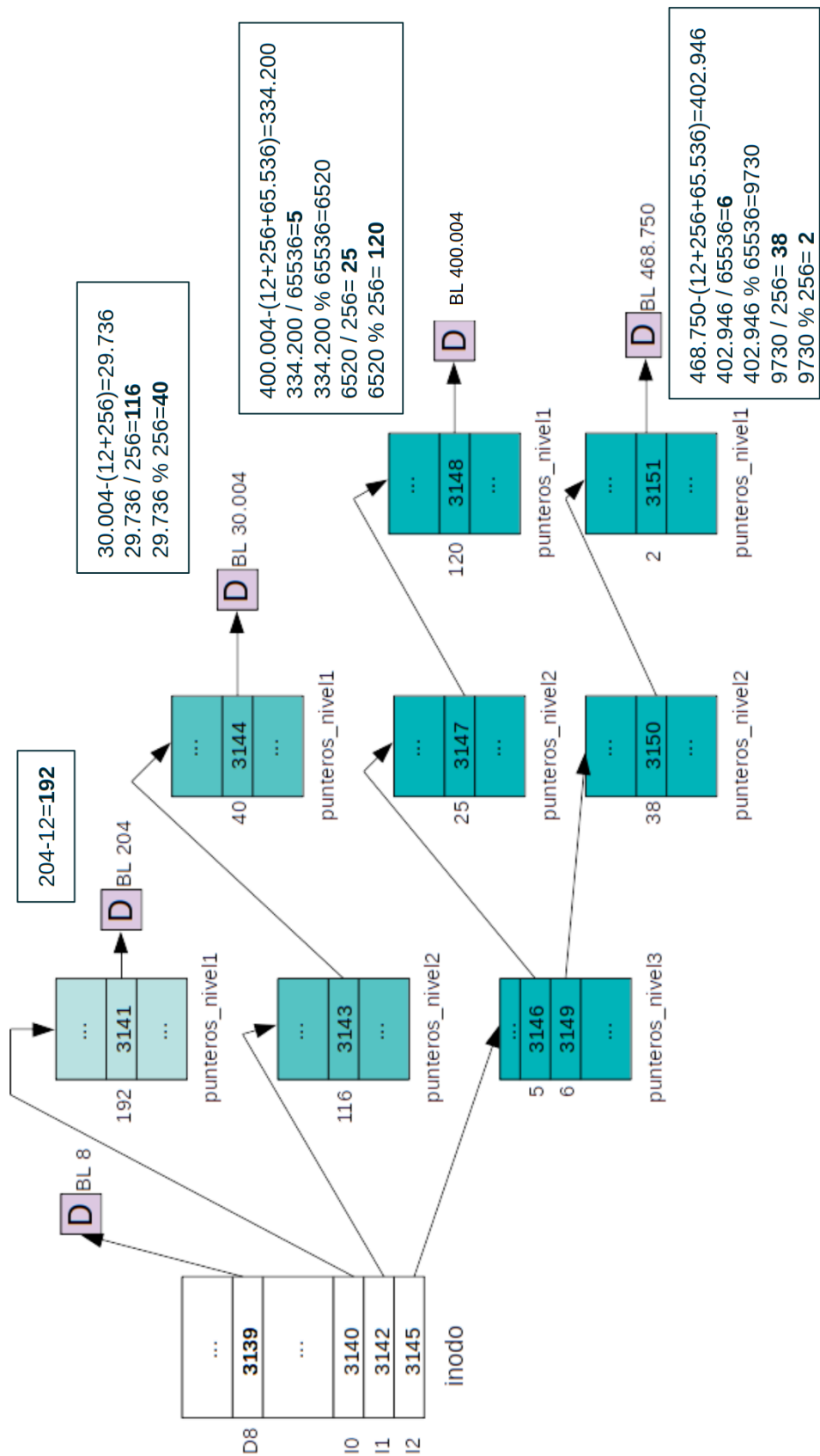
Nivel 4

`ficheros_basico.c` {traducir_bloque_inodo(),
obtener_indice(), obtener_nRangoBL()}

disco	metadatos (SB, MB, AI)	3139
	[BL8]	3140
	punteros_nivel1 BL 204	3141
	[BL204]	3142
	punteros_nivel2 BL 30.004	3143
	punteros_nivel1 BL 30.004	3144
	[BL30.004]	3145
	punteros_nivel3 BL 400.004	3146
	punteros_nivel2 BL 400.004	3147
	punteros_nivel1 BL 400.004	3148
	[BL400.004]	3149
	punteros_nivel2 BL 468.750	3150
	punteros_nivel1 BL 468.750	3151
	[BL468.750]	
	⋮	
		99999

Observaciones:

- Los **bloques de datos** se reservan pero los salva `mi_write_f()` de la capa superior, tras escribir en ellos.
- Si en un **bloque de punteros** modificamos un índice en el buffer de memoria principal, hay que salvarlo en el dispositivo.
- El **inodo** lo salva `traducir_bloque_inodo()`, sólo si se han modificado campos del **inodo** (punteros y `numBloquesOcupados`).
 - Así evitaríamos un *big lock* en `mi_write_f()` con concurrencia.
 - Requiere pasar `ninodo` como parámetro en `traducir_bloque_inodo()` en vez de pasar la estructura del inodo por referencia.



Anexo 1: Enfoque descomposición polinomial para el cálculo de índices

Abordaremos el problema desde un enfoque de los sistemas de numeración.

Primeramente restamos los punteros a bloques de datos anteriores al dado para centrarnos sólo en los que se pueden apuntar en la ramificación a la que pertenece el BL dado (enumerándolos así desde el 0):

- $nRangoBL=1 \rightarrow nblogico - DIRECTOS$
- $nRangoBL=2 \rightarrow nblogico - INDIRECTOS0$
- $nRangoBL=3 \rightarrow nblogico - INDIRECTOS1$

Ej: $400.004 - 65804 = 334.200$. Eso significa que el BL 400.004 con respecto al conjunto global de bloques lógicos del inodo es el mismo que el BL 334.200 si lo relativizamos a los 256^3 posibles que saldrían del despliegue de I2, empezando a contar desde 0.

Si NPUNTEROS=256 entonces podríamos enfocar el problema de calcular los índices como si fuera hallar la descomposición polinomial, de ese BL relativizado, en un sistema de numeración de base 256.

Vamos a ver primero cómo es la descomposición polinomial de un número en base decimal (10) y cómo la obtenemos, por ejemplo para el número 1234:

$$1234 = 1 \cdot 10^3 + 2 \cdot 10^2 + 3 \cdot 10 + 4 = 1000 + 200 + 30 + 4$$

De donde salen los multiplicandos de las potencias de 10 utilizando / y %:

Millares	$1234 / 10^3 = 1$	$1234 / 10^3 = 1$
Centenas	$1234 \% 10^3 = 234$ $234 / 10^2 = 2$	$(1234 \% 10^3) / 10^2 = 2$
Decenas	$234 \% 10^2 = 34$ $34 / 10 = 3$	$(1234 \% 10^2) / 10 = 3$
Unidades	$1234 \% 10 = 4$	$1234 \% 10 = 4$

Y ahora vamos a trasladar este enfoque polinomial a los índices de los bloques de punteros de un inodo. En este caso la base será **NPUNTEROS**, es decir 256 en el ejemplo:

$$334.200 = 5 * 256^2 + 25 * 256 + 120 = 327.680 + 6.400 + 120$$

De donde salen los multiplicandos de las potencias de 256 utilizando / y %:

Índice nivel 3	$334.200 / 256^2 = 5$	$334.200 / 256^2 = 5$
Índice nivel 2	$334.200 \% 256^2 = 6.520$ $6.520 / 256 = 25$	$(334.200 \% 256^2) / 256 = 25$
Índice nivel 1	$6.520 \% 256 = 120$	$(334.200 \% 256^2) \% 256 = 120$

Estas tres expresiones de la última columna son las que implementa `obtener_indice()` para el caso `nRangoBL=3`.